

Software Engineering COMP1035

Lecture 01

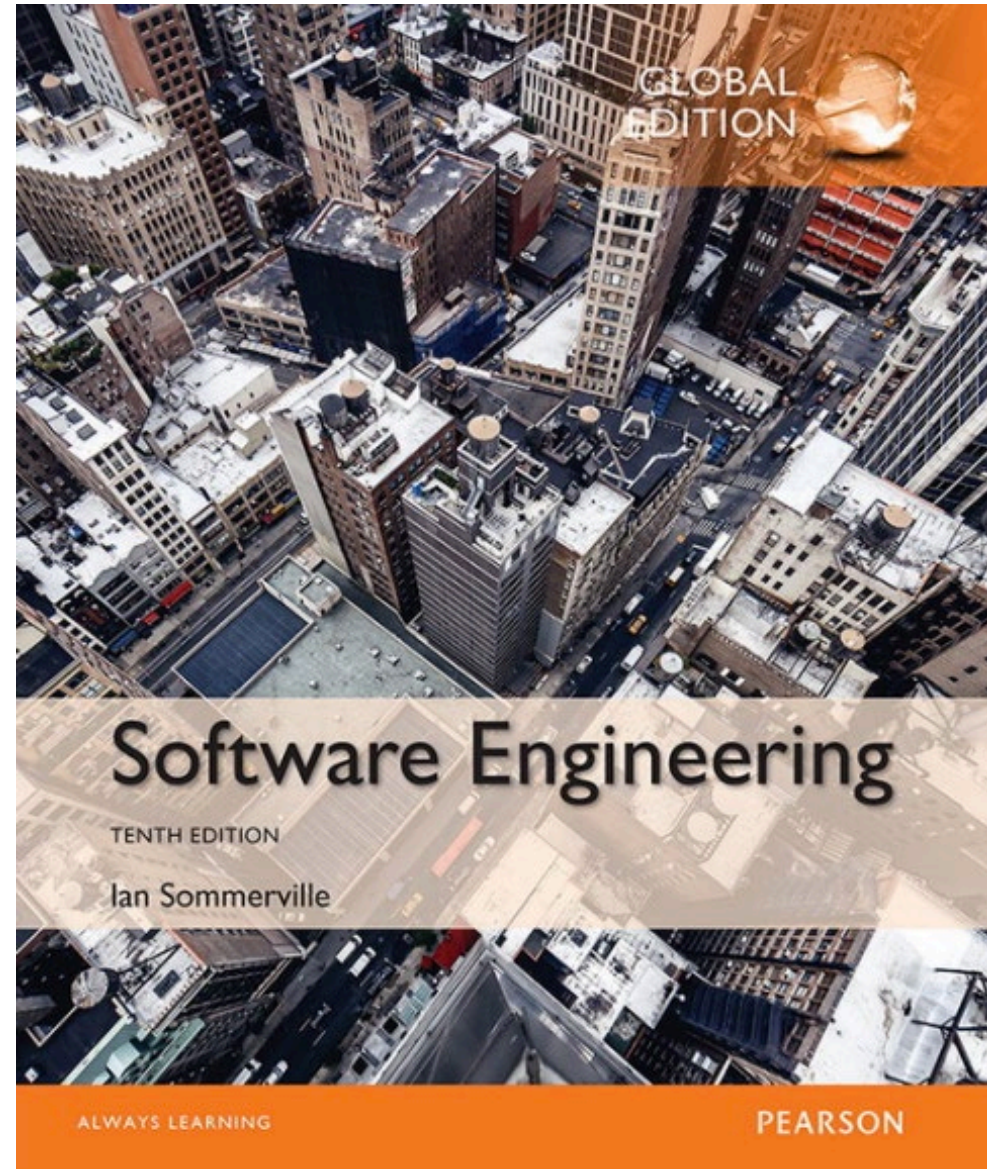
Introduction



Plan for the Module



SE Book



Labs & Assessments

- 50% Exam + 50% Coursework.
- 2-hours Lecture per week typically on **Friday at 9pm to 11pm.**
- 2-hours Lab session per week typically on **Friday 1pm to 3pm.**
 - 8 Skill Labs – to prepare you for subsequent assessment.
- Two coursework: CW1 (20%) and CW2 (30%)



Module Convenors

- Dr. Xinan, Chen
 - **Email:** Xinan.Chen@nottingham.edu.cn
 - **Office:** PMB442
 - **Consultation Hours:** Tuesday @ 11:00-13:00
- Dr. Yanwen Mao
 - **Email:** Yanwen.Mao@nottingham.edu.cn
 - **Office:** PMB444
 - **Consultation Hours:** Friday @ 12:00-13:00 & 15:00-16:00

What Type of
Learning are
We Doing in
this Module?



Why are We Working in
Group?

Working in Group

- Software Engineering happens in groups.
 - Y3 software engineering group project (GRP) is done in groups.
- You'll learn more by discussing these topics.
 - Because they are all about intellectual comprehension.
- Every lab is groupwork:
 - Peer-assess each others contribution, at end of each lab.
 - And your own contribution.

Working in Group

- Group of FIVE or SIX students.
- You can submit preferred/agreed group application using link in Moodle.
 - By **Wednesday 21st Feb. 2023 at 12 noon.**
 - Final group list will be released before Lab 1.
- Small groups with less than FIVE or SIX students will be merged with other small groups.

Time Allocation

- 20 hours of lectures (including revision).
- 20 additional hours of extra reading (e.g. from the book).
- 20 hours for labs + 10 hours extra for last coursework.
- 10 hours of preparation for labs and peer making.
- 20 hours of revision & taking exam.

Using Moodle

- The schedule of classes is on Moodle.
- The class materials will be on Moodle.
- Q&A Forum for questions.
 - Moodle discussion forum.
 - Q&A via Microsoft Forms.
- Coursework will be released on Moodle.
- We'll be using announcement mechanism in Moodle to provide updates to you.

Labs

- Labs are mandatory.
- There are tasks to prepare you for the coursework.
- Encourage all groupmates to work on the lab exercises together.

Using CSProjects (GitLab)

- You will create a GitLab repository FOR YOUR GROUP.
- You will have a template repository forked in it. (Lab 01)
- All submitted work will be the group's work.
- We will mark based on your group's work.
- Your group repository is basically what you are “managing” for the semester.

Aims of the Module

What is Module Specification?

- Summary of content: You'll be introduced to the concept of Software Engineering and will be taken through the software development process:
 - deciding exactly what should be built (**Requirements**),
 - designing how it should be built (**Software Specification/Architecture**),
 - development strategies (**Implementation & Testing**), and
 - maintaining change (**Software Evolution and Maintenance**).
- An aim of this module is to provide a general understanding of Software Engineering;
 - Typical phases of the **software lifecycle** with reference to practical Requirements and Specification, Software Design, and Implementation & Testing techniques.
 - To prepare you for the various software development projects undertaken throughout your studies.

COMP1035 (FSE) Aims

1. Teach you an ***overview** of the **whole** Software Engineering process.
2. Give you initial **practical experience** of work at different stages of the process.
3. **Prepare you** for Software Engineering group project in Part 1 (Y3).

(Note: Not able to cover everything in a 10 credits module)

What is the “Software” We are Talking About?

What is Software?

- Such that it needs a big process.
- You largely did not build software in semester 1 – you wrote code.
- Bigger software examples:
 - A phone game app, with a global scoreboard
 - Twitter software
 - Google Chrome
 - Universal Credit System for Benefits (for Government)

Software?

- Last semester, you built dedicated scripts to achieve one thing!
- A **component** of software.
- Not ready for 'real human' use.
- Probably was buggy.
- Software is something that is working, usually for 'real humans' to use to help them do something.

Scripts

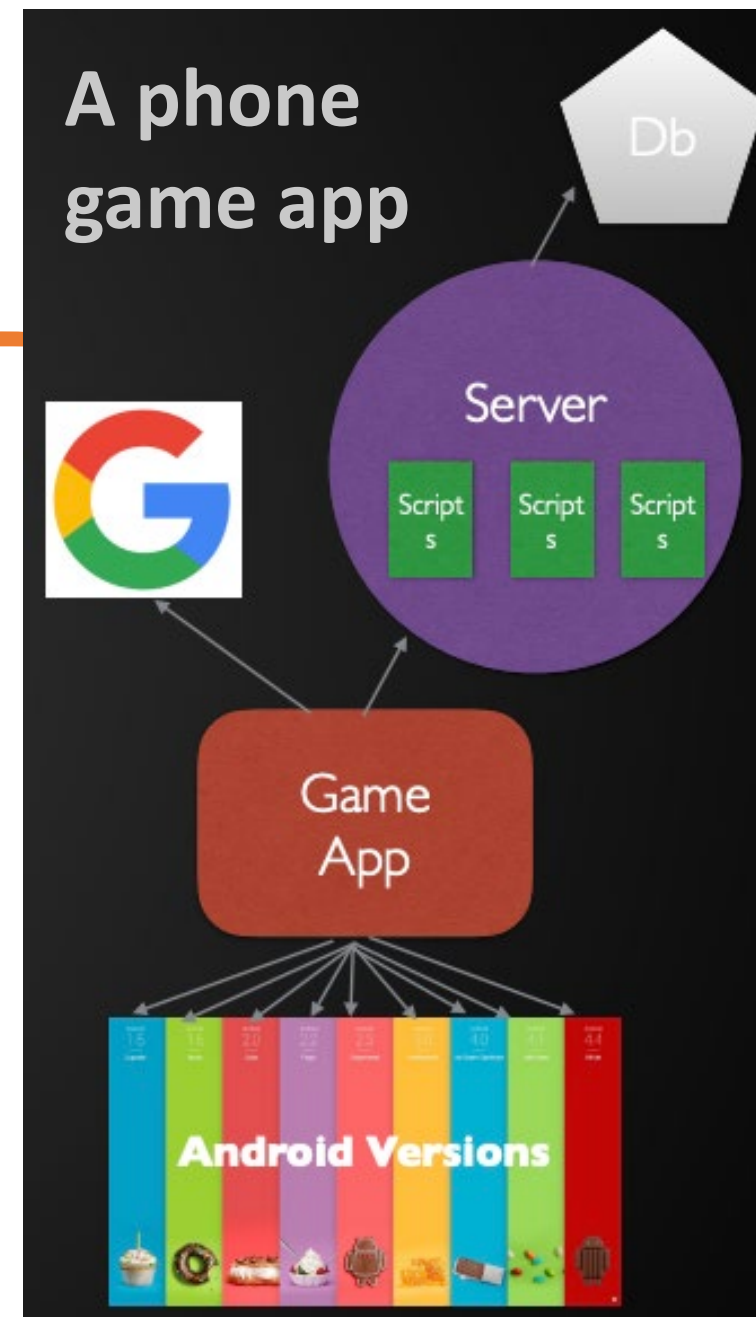
Scripts

Scripts

Scripts

Actual Software?

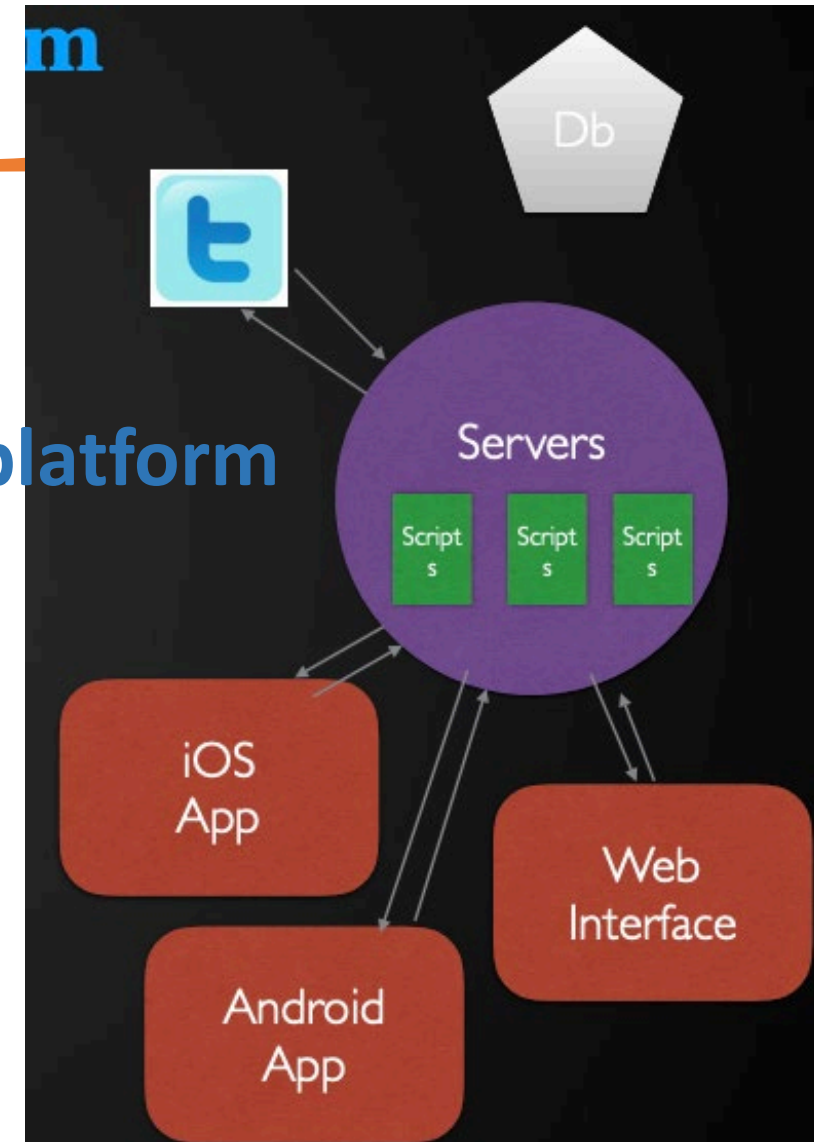
- Ready for real users.
- Integrates many features in the game.
- Might integrate with other services (e.g., login).
- Might have a server component.
 - Talking to a database.
- Need to push out multiple versions.



What is Software?

- Let's ignore issues of scale for now.
 - Pretend '1 database' is ok.
- Basically, a server driven platform.
- Has a lot interfaces to it.
 - Each is a piece of software.
 - Each one integrates functions.
 - Each piece might have different bugs.
- New features rolled out across them.
- Video of Twitter Scale:
 - <https://www.infoq.com/presentations/Twitter-Timeline-Scalability/>

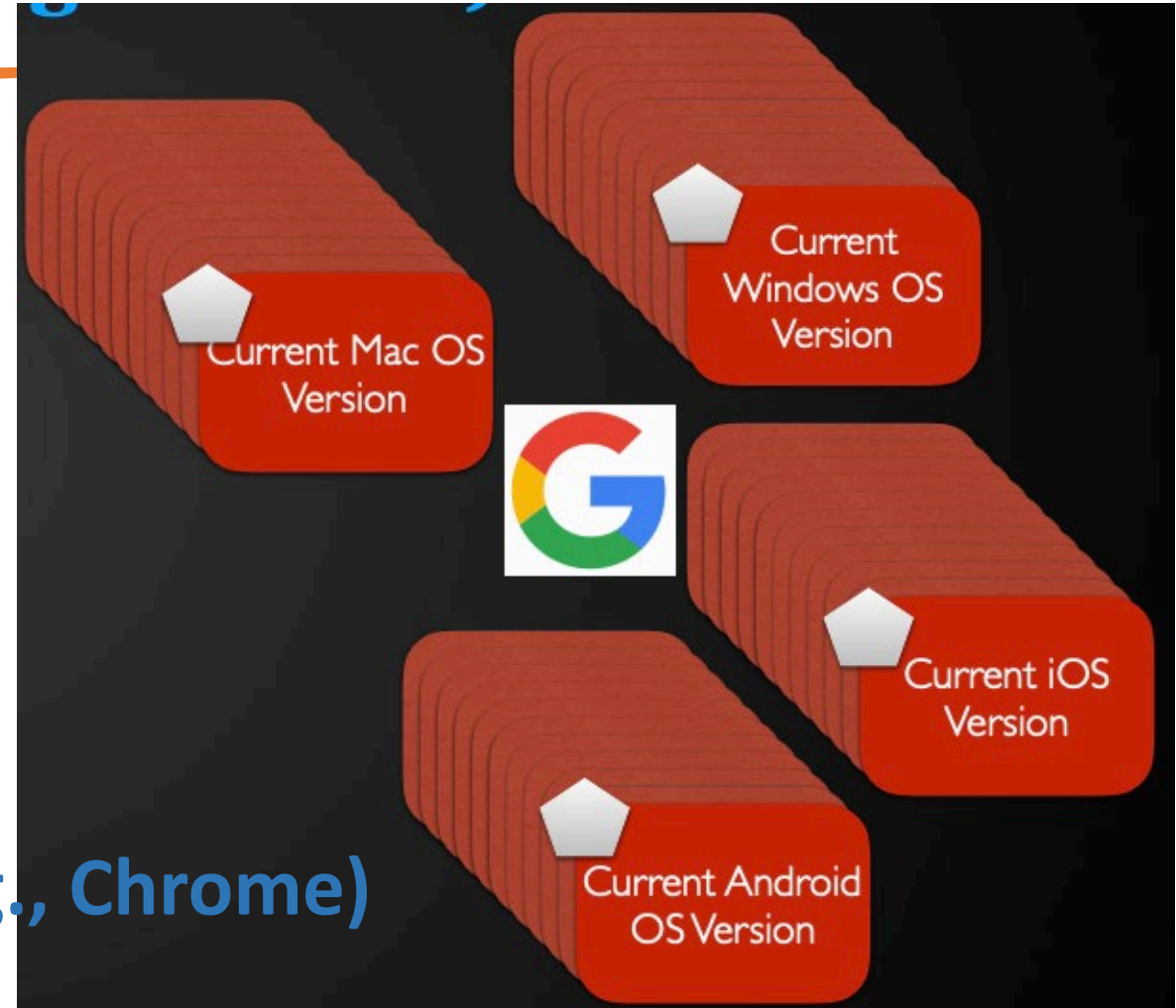
A social media platform



What is Software?

- Integrate multiple functions.
- Has a version history on every platform.
- New functions roll out on all platforms.
- All authenticate and share data.

A web browser (e.g., Chrome)



UoN Campus Solution (Simplified)

- In 2015 – UoN wants to buy a software package to handle student records – merge 4 different systems
 - Student records.
 - Module enrolment.
 - Admissions.
 - Course documentation.
- To use on all 3 campuses (UK, Malaysia, Ningbo China).
- They have brought this from a software company.

UoN Campus Solution (Simplified)

- **Phase 1** – Admission starts across all 3 sites in 2016.
- **Phase 2** – Went live in Malaysia in January 2017.
- **Phase 3** – Full development due January 2018 – Ningbo China goes ahead with all data managed in both old and new systems.
 - UK delayed till April 2018.
 - UK delayed till Summer 2018 (still managing data in both old & new).
 - UK delayed until Christmas 2018.
 - January 2019 – Went live in UK (data still managed in old system, just in case).
- **Phase 4** – Finance part of the system going ahead in 2019/20!

Testing of Campus Solutions

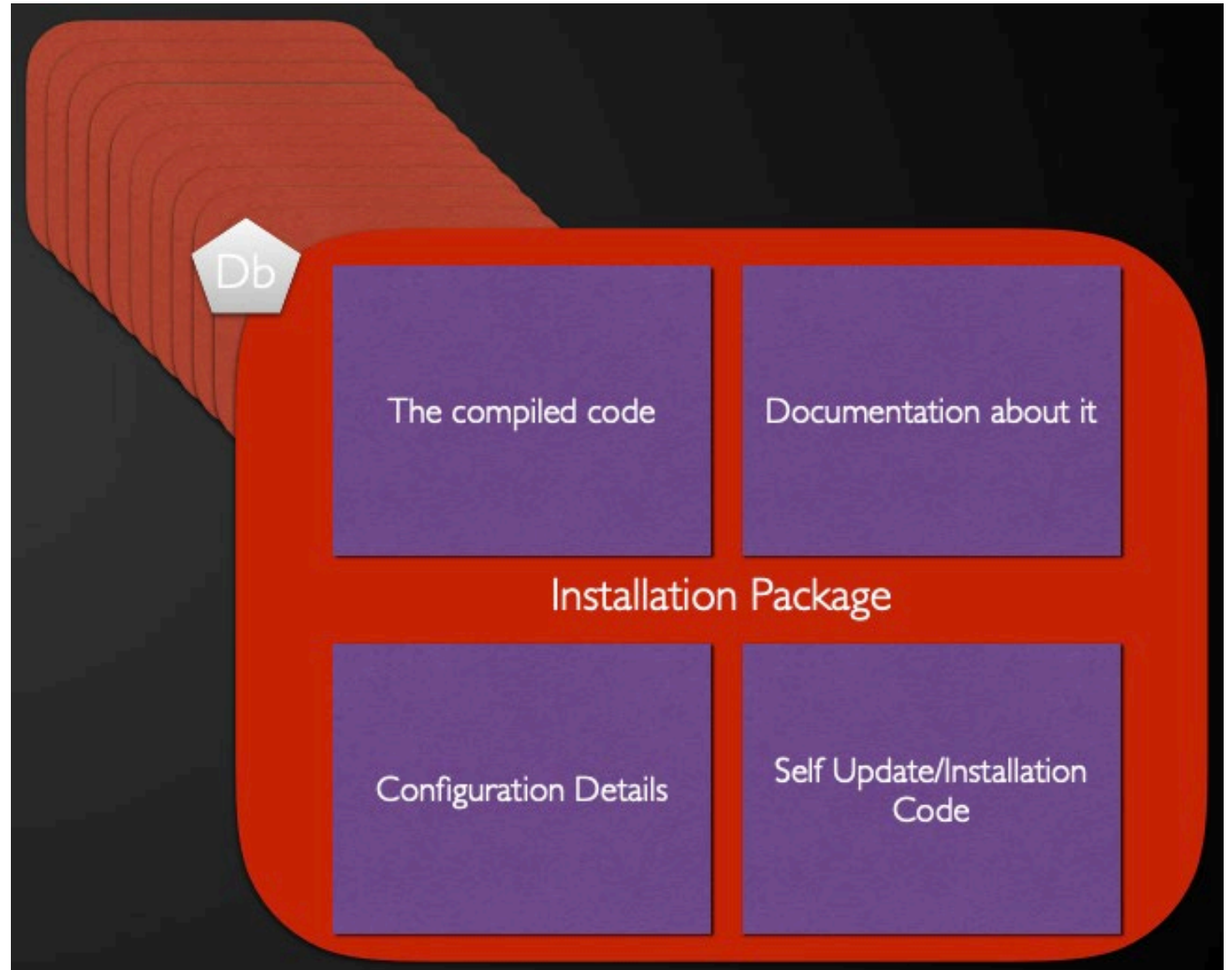
- <http://blogs.nottingham.ac.uk/campussolutions/2018/10/24/overview-of-campus-solutions-testing-procedure/>

Here is an insight into the comprehensive testing steps that the team are working through:

1. **Unit Testing** – In this initial test, the code, configuration, Oracle functions, data handling and security of the functionality are reviewed and tested by the developer.
2. **Functional Acceptance Testing (FAT)** – This involves analysing the original requirements and running tests to make sure each of them has been delivered by the developer.
3. **Business Acceptance Testing (BAT)** – After the functionality has passed FAT, it is given to the business user for them to run tests on how the process will be run. This is where the business checks that the functionality works as the University expects it to and nothing has been missed.
4. **End-to-end testing** – 15 different student types across the student lifecycle have been identified at the University. In this test, the functionality is tested across as many varied circumstances as possible to ensure it works with every student type. Connections and data transfers to other University systems are also tested here.
5. **Regression testing** – As new functionality is brought into the system, it is important to ensure that existing functionality continues to work. This testing stage ensures that new and existing functionality works in harmony together.
6. **Performance testing** – This assesses the performance of the functionality in terms of speed, scalability and stability. Performance testing ensures the system can cope with events such as confirmation and clearing and start of session when system usage is very high.
7. **Penetration testing** – It is important to ensure that the functionality and the system as a whole has the best security standards to withstand any malicious attacks, such as viruses or worms. The servers and network that the system will be running on are also tested. Specialist external companies are used to 'attack' the system and provide recommendations to improve security.

What is Software?

- Software includes
 - The compiled **code**.
 - **Documentation**.
 - Configuration components.
 - Installation/upgrade.
- All in an installer that 'deploys'.



What is Software Engineering?

“The application of a **systematic, disciplined, quantifiable** approach to the **development, operation** and **maintenance** of software [...];

That is, the application of engineering to
software”

- Wikipedia

(actually, gives a good definition, despite being a bad reference source)



The Need for Software Engineering

Bad Software is Frustrating



Sarah James @SarahJames456

1h

Phew!

Finally got WordPress working(ish) & I've entered my 1st food blog challenge.

What a **frustrating** day.

sarahjamesonline.com/?p=24 🌐

Expand



Katerina @brosternova

5h

Trying to teach my mum about how to copy and paste stuff on a **computer...** I could actually kill myself -_- so **frustrating!!!**

Expand



olivia sian @Olivia_Sian

25 Sep

I really hate Central's email. It never loads for my **computer** or shows my attachments. **#frustrating** ☹☹

Expand



Honor @On_aspiderwebb

23 Sep

I know! I'll call my company HonorSoft and develop **frustrating computer** software

Expand

Bad Software is the Worst



The image is a collage of six tweets, each with a profile picture, name, handle, date, text, and an 'Expand' link. The tweets are arranged in a staggered, overlapping layout on a dark background.

- Niscenti** @Niscenti 27 Sep
iTunes, the **worst software** I've ever used. It makes me so angry.
Expand
- Kyle Noble** @Noble2045 24 Sep
H&R Block has to be the **worst tax software** ever
Expand
- We Are The Tay** @WeAreTheTay 28 Sep
The **software** for the Galaxy s4 to connect with a Mac is the **worst** thing. The **worst** thing.
Expand
- sandeep** @sandy135is 14h
Adobe Premier is the **worst Software** i ever seen
Expand
- Christian Gloddy** @gloddy 8 Aug
TV manufacturers create beautiful screens and then pair it with the **worst software** you could possibly imagine.
Expand
- Matthew Calcara** @matthewcalcara 19 Aug
It's a tight race between Windows & Flash for the title of "**Worst Software** Ever Invented by Humankind" - alvinalexander.com/internet/flash...
Followed by HootSuite
Expand



Bad Software is Frustrating

"Unexpected item in bagging area.
Please wait for assistance."

The Software Crisis (Nato, 1968)

- Software was bad, or worse.
- Software was unreliable behind schedule and cost more than expected.
- 'Software Engineering' was coined to understand the 'making' of software.
- During the 1970s/80s (and still now) people develop new processes to better 'engineer' software.

“The average customer of computing industry has been served so poorly that he expects his system to crash all the time, and we witness a massive world wide distribution of bug-ridden software for which we should be deeply ashamed.”

- Dijkstra, 2001



Ongoing SE Project Failurees

- The global cost of IT failure was estimated at +6 trillion dollars in 2009. (source: Roger Sessions, The IT Complexity Crisis: Danger and Opportunity)
- Only 32% of software projects were “successfully completed” (i.e., on time, on cost, and with expected functionality) in 2009. (Source: Standish CHAOS 2009 Update)
- Only 16% of software projects were successfully completed in the UK in 2009. (Source: British Computer Society)
- “A failing industry ...
 - If building engineers build buildings with the same care as software engineers build systems, the first woodpecker to come along would be the end of civilization as we know it.” (Source: Paul Dorsey, Top 10 Reasons Why System Projects Fail)

Ongoing SE Project Failures

https://en.wikipedia.org/wiki/List_of_failed_and_overbudget_custom_software_projects



The screenshot shows the Wikipedia page for 'List of failed and overbudget custom software projects'. The page includes a sidebar with navigation links, a main content area with a warning about incomplete lists, and a table of contents.

WIKIPEDIA
The Free Encyclopedia

[Main page](#)
[Contents](#)
[Featured content](#)
[Current events](#)
[Random article](#)
[Donate to Wikipedia](#)
[Wikipedia store](#)

Interaction
[Help](#)
[About Wikipedia](#)
[Community portal](#)
[Recent changes](#)
[Contact page](#)

Tools
[What links here](#)
[Related changes](#)
[Upload file](#)
[Special pages](#)
[Permanent link](#)
[Page information](#)

Article [Talk](#)

[Read](#) [Edit](#) [View history](#)

List of failed and overbudget custom software projects

From Wikipedia, the free encyclopedia

This article contains one or more [incomplete lists](#) which may never be able to satisfy particular standards for completeness. You can help by [expanding it](#) / [them](#) with entries that are [reliably sourced](#).

This is a list of notable [custom software](#) projects which have significantly failed to achieve some or all of their objectives, either temporarily or permanently, and/or have suffered from significant [cost overruns](#). For a list of *successful* major custom software projects, see [Custom software#Major project successes](#).

Note that failed projects, and projects running over budget, are not necessarily the sole fault of the employees or businesses creating the software. In some cases, problems may be due partly to problems with the purchasing organisation, including poor [requirements](#), over-ambitious requirements, unnecessary requirements, poor [contract](#) drafting, poor contract management, poor end-user [training](#), or poor operational management.

Contents [\[hide\]](#)

- [1 Permanent failures](#)
- [2 Temporary issues and mere budget overruns](#)
- [3 Projects with ongoing problems](#)
- [4 See also](#)
- [5 References](#)
- [6 External links](#)

Permanent failures [\[edit\]](#)

NEWS

[Home](#)[UK](#)[World](#)[Business](#)[Politics](#)[Tech](#)[Science](#)[Health](#)[Education](#)[Entertainment](#)

Technology

US prisoners released early by software bug

🕒 23 December 2015 | [Technology](#)

US Prison Bug

- More than 3,200 US prisoners have been released early because of a software glitch.
- The bug miscalculated the sentence reductions prisoners in Washington state had received for good behavior.
- Analysis of the errors showed that, on average, prisoners whose sentences were wrongly calculated got out 49 days early. One prisoner had his sentence cut by 600 days.
- An update that applies the correct formula for calculating sentence cuts is due to be placed by 7 January.

Hawaii Incident

- A test of “incoming nuclear bomb” message was accidentally distributed as a live message to citizens.
- A human error, but the software has been blamed for facilitating it.
- Requiring as little as 3 clicks either way, with the same UI
 - The interaction for test vs live is the same – with the same popups.
 - Depending on a user ‘noticing’ their error.

Ariane 5 Rockets

- “Ariane 5's first test flight (Ariane 5 Flight 501) on 4 June 1996 failed, with **the rocket self-destructing 37 seconds after launch because of a malfunction in the control software**. A data conversion from 64-bit floating point value to 16-bit signed integer value to be stored in a variable representing horizontal bias caused a processor trap (operand error) because the floating-point value was too large to be represented by a 16-bit signed integer.
- The **software was originally written for the Ariane 4** where efficiency considerations (the computer running the software had an 80% maximum workload requirement) led to four variables being protected with a handler while three others, including the horizontal bias variable, were left unprotected because it was thought that they were "physically limited or that there was a large margin of error".
- The **software, written in Ada, was included in the Ariane 5 through the reuse of an entire Ariane 4 subsystem despite the fact that the particular software containing the bug**, which was just a part of the subsystem, was not required by the Ariane 5 because it has a different preparation sequence than the Ariane 4.”

More Contemporary Example

- <https://twitter.com/allenholub/status/1145006228348657664?s=12>

The screenshot shows a Twitter interface with a sidebar on the left containing navigation links: Home, Explore, Notifications, Messages, Bookmarks, Lists, Profile, and More. A blue 'Tweet' button is at the bottom of the sidebar. The main content area displays a thread by Allen Holub (@allenholub) titled "Boeing's 737 MAX software outsourced to \$12.80-an-hour engineers" (with a link to smh.com.au/business/compa...). The tweet text reads: "You get what you pay for." Below the text is a photo of a Boeing 737 MAX aircraft. The tweet is retweeted by 'smh.com.au' with a description: "Boeing's 737 MAX software outsourced to \$12.80-an-hour engineers. The software blamed for the Lion Air and Ethiopian Airlines disasters was developed at a time Boeing was laying off experienced engineers and cutting costs." The timestamp is "5:28 PM · Jun 29, 2019 · TweetDeck". On the right, the 'Relevant people' section shows Allen Holub's profile with a 'Follow' button. The 'United Kingdom trends' section lists: 1. Trending #MondayMotivation (48.5K Tweets), 2. Trending #HolocaustMemorialDay (trending with #HMD2020, #standtogether), 3. Trending #Auschwitz75 (trending with Auschwitz-Birkenau, On the 75th), and 4. Politics - Trending #HolocaustRemembranceDay (trending with Primo Levi).

Software Maintenance

New Scientist uses [cookies](#) to provide you with a great user experience. By using this website, you agree to the [use of cookies](#) on your device. [Accept](#)

THE DAILY NEWSLETTER
Sign up to our daily email newsletter

NewScientist

News [Technology](#) Space Physics Health Mind Environment More ▾ [Shop](#) [Tours](#) [Events](#) [Jobs](#) [Site Access](#) [Search](#)

A lazy fix 20 years ago means the Y2K bug is taking down computers now

[f](#) [t](#) [w](#) [in](#) [re](#) [m](#) [p](#)

TECHNOLOGY 7 January 2020
By [Chris Stokel-Walker](#)



Advertisement

TRENDING	LATEST	VIDEO	FREE
What are the symptoms of the new coronavirus and how deadly is it?	1		
One in 16 US women were forced into having sex for the first time	2		
A radical idea suggests mental health conditions	3		

Top 10 SE Mistakes

1. Presume 'good code' is the only thing that matters.
2. Aim to finish at the delivery deadline.
3. Don't design a data model, let the code produce data it needs.
4. Use a Technical Lead that has never built a similar system, rather than choose/hire someone who has.
5. Hire forty developers to make coding go faster.
6. Build the system in the language you know best, rather than 'the best tool for the job'.
7. Hire a junior developer to handle the migration.
8. Skip the testing phase because the project is behind schedule.
9. Change code, without planning, to meet newly discovered requirements.
10. Buy a commercial, off-the-shelf package and customise it ... a lot.

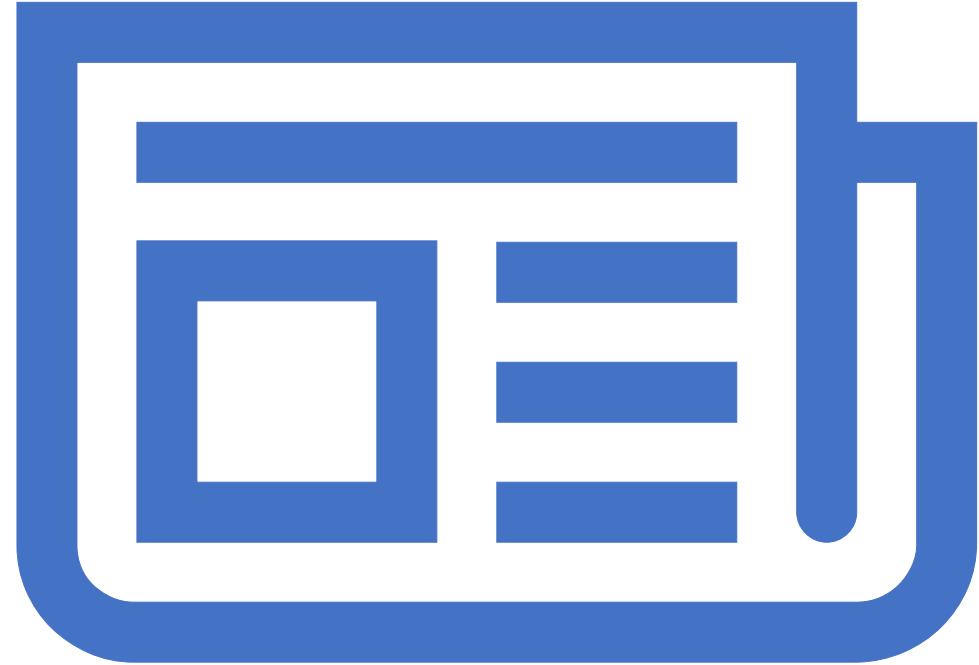
Dorsey's Conclusions

1. Don't cut corners, methodologically. In the long run, this result in system failure or an inadequate system that doesn't meet the users' needs.
2. **Audit each major deliverable and step along the way for accuracy and correctness.**
3. Carefully monitor top management support for the project. Make sure that managers are aware of the progress of the team.
4. Secure the correct technical lead for the project.

“There is no free lunch in software engineering. If you insist on **keeping costs low** and **hurrying the project along**, then **quality will be low** or the **risk of failure will be high** no matter how well the project is managed.”

- Paul Dorsey

Overview of SE Process



Software Engineering Process

- Software engineering is an engineering discipline that is concerned with **all aspects** of software production.
- “Software engineering is concerned with **principles** and **methods** for **specifying, designing, implementing** and **maintaining** large software systems.” R.Sirewalt 2004
- Software engineers should adopt a **systematic and organized approach** to their work and **use appropriate tools** and techniques **depending on the problem** to be solved, the development constraints and the resources available.

Software Engineering Process

- You are not building something **for** yourself
 - you need to understand/agree what someone else wants
- You are not building something **by** yourself
 - you need to work with others - in different countries even
- You are not building it for people **like** yourself
 - it has to work for 'real humans', not computer experts
- You are not working on code you **wrote** yourself
 - you need to figure out how the hell is this stupid code working, who wrote this? an octopus? its the worst code I've ever seen. were they drunk? There's no comments.



Programming Wisdom
@CodeWisdom

Following

"Any code of your own that you haven't looked at for six or more months might as well have been written by someone else." - Eagleson's law

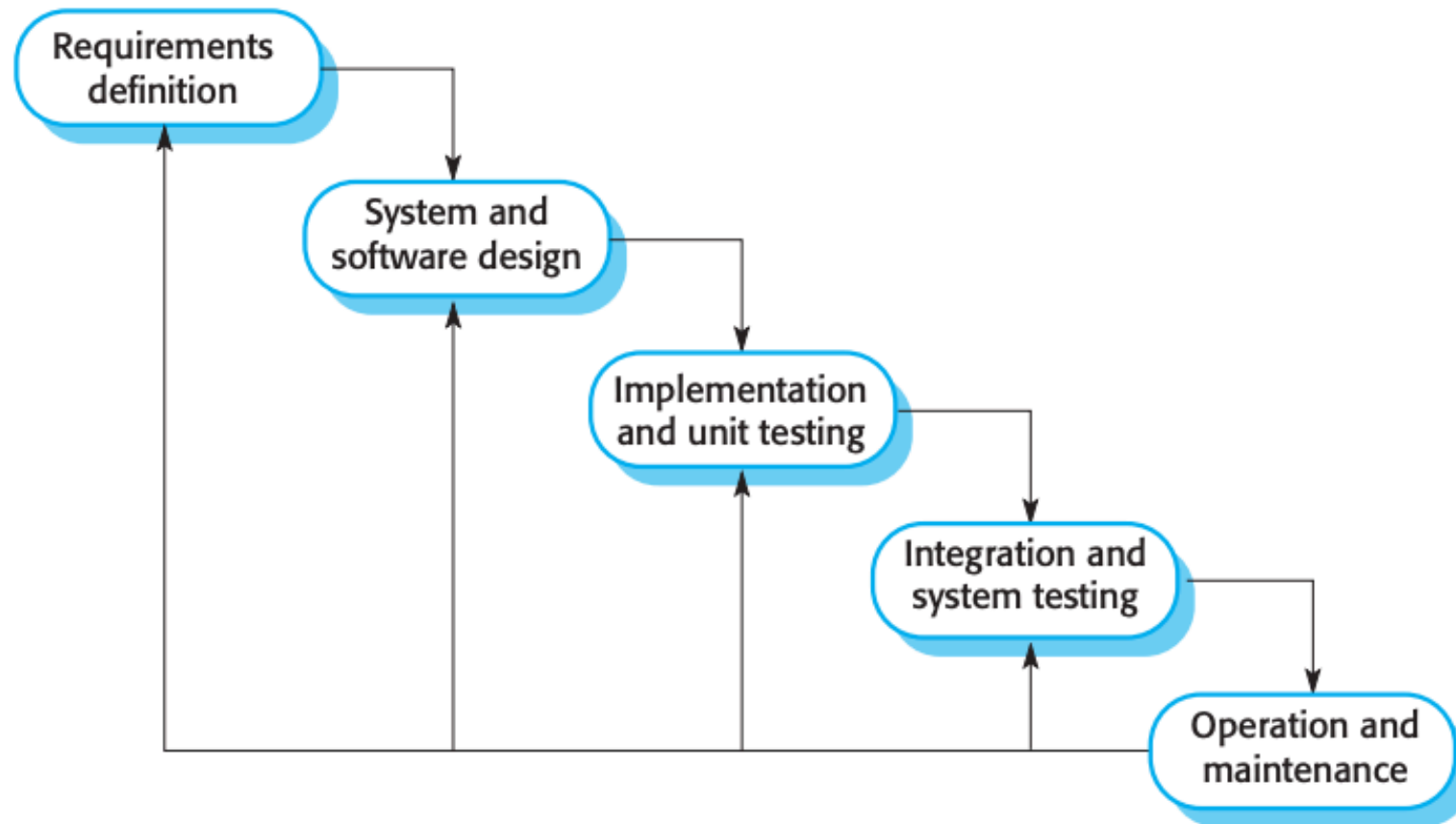
6:00 PM - 4 Jan 2019

Core SE Process Stages

- Generic activities in all software processes are:
 - **Specification** – what the system should do and its development constraints.
 - **Development** – production of the software system.
 - **Validation** – checking that the software is what the customer wants.
 - **Evolution** – changing code in response to demands.
- Utilising different SE models that cover the SE processes. Each has its own merits and limitations, and the usage is depending on the project's suitability.
 - Waterfall Model
 - V-Model
 - Iterative Model

Waterfall Model

- Developed in 70s



- http://sqa.fyicenter.com/FAQ/Software-DevelopmentModels/Software_Development_Models_V_Model.html

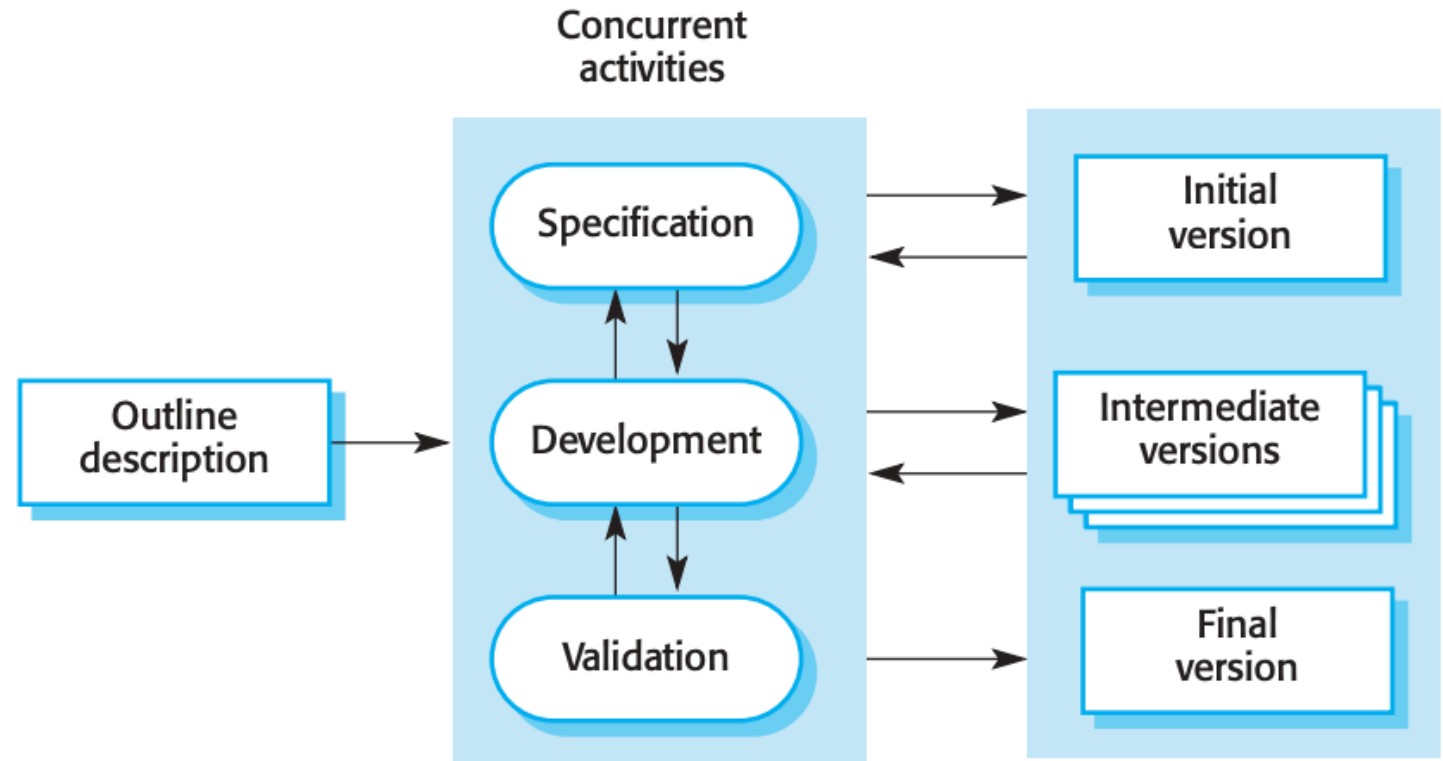
Developer's life Cycle

```
graph TD; subgraph Verification_Phase [Verification Phase]; BRS[Business req. Specification] --> SRS[System Req. Specification]; SRS --> HLD[High level Design]; HLD --> LLD[Low level Design]; LLD --> Coding[Coding]; end; subgraph Validation_Phase [Validation Phase]; Coding --> UT[Unit Testing]; UT --> CT[Component Testing]; CT --> SIT[System Intergration Testing]; SIT --> AT[Acceptance Testing]; end; BRS -.-> AT; SRS -.-> SIT; HLD -.-> CT; LLD -.-> UT;
```

Problems with Waterfall & V

- Needs stable and “perfect” requirements.
- Can’t always anticipate what you are going to have to do.
- Does not account for revision or refactoring.
- Too inflexible and static.
- Depends on getting each stage exactly right
 - Changes can have many knock-on effects.

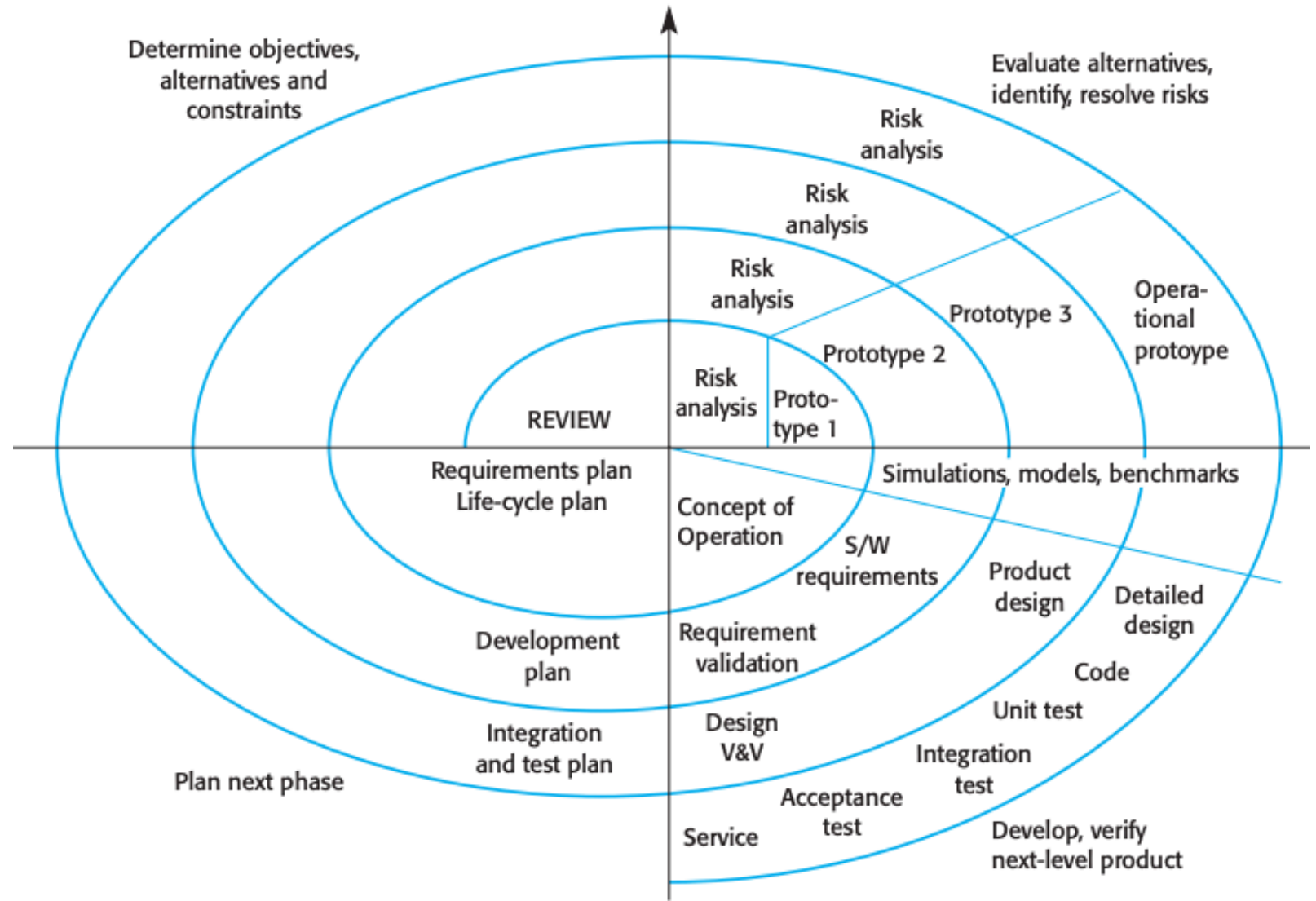
Iterative Model



The iterative process model is a **software development life cycle (SDLC)** approach where **development begins with clearly defined initial requirements**. Additional features are then incorporated into the base software product through iterative cycles until the final system is completed.

Iterative Model – Boehm's Spiral

Developed in late 80s



Problems with Iterative Model

- Lack of process visibility.
 - Systems are often poorly structured.
- Too much doing and not enough planning.
- Special skills (e.g., in languages for rapid prototyping) may be required.
- Lightweight documentation taken to mean no documentation.

Effort in SE Processes

- 60% in designing and building the software.
- 40% in testing/delivering the software.
- BUT – Maintaining/Evolving software often costs **more** than producing it in the first place.

Summary

- Several real-world cases of SE software failures.
- Importances of the SE methodologies and brief introduction of the overview of SE models.
- 4 stages of SE activities:
 - Requirements and Specification
 - Development (will be covered in other programming modules)
 - Validation
 - Evolution (will be covered in COMP2059 Developing Maintainable Software)
- Why do we need SE process?
 - Coding is one stage, out of e.g., 5 or more stages.
 - Having an appropriate SE Process/Methodology is really important.
 - Quality means getting the right process - and process right.
 - Quality means getting every stage done right, not fast.

Optional Readings

- Major Software Fails – Wikipedia
- Universal Credit Fail – 2013
- Universal Credit Fail – 2016
- Facebook Software Architecture
- Video Lecture – How Twitter is Built (c2013)

THANK

YOU